

Chapter 4: Web Services & APIs

By Er. Ukesh Thapa

Syllabus

4.1 API basics: Role in web applications

4.2 REST principles and design, RESTful APIs

4.3 JSON versus XML data exchange

4.4 Data validation and serialization

4.5 Microservices

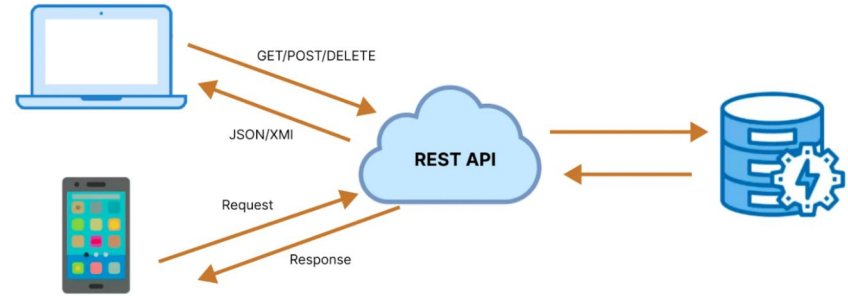
Total Marks: 9

Application Programming Interface (API)

- ❖ API stands for Application Programming Interface
- ❖ They enable applications and systems to communicate and share data efficiently.
 - Enable seamless interaction between applications, platforms, and services.
 - Power everyday features such as weather updates, payments, and e-commerce checkouts.
- ❖ Why we need API:
 - **Reusability**: Avoid reinventing the wheel by leveraging existing APIs (e.g., Google Maps API, Stripe Payments API).
 - **Efficiency**: Save development time by integrating ready made functionalities.
 - **Scalability**: Enable modular, distributed systems that can grow easily.
 - **Integration**: Connect multiple platforms web, mobile, IoT, or analytics.
 - **Automation**: APIs allow machines to talk to machines without manual input.

APIs Working

- ❖ **Request:** The client sends a request to an API endpoint (URI).
- ❖ **Processing:** The API forwards the request to the server.
- ❖ **Response:** The server processes and sends back the requested data.
- ❖ **Delivery:** The API returns the server's response to the client.



API Architectures

❖ **REST (Representational State Transfer):**

- REST is a simple, flexible API architecture that uses HTTP methods (GET, POST, PUT, DELETE) for communication.
- Data Format: JSON, XML.

❖ **SOAP (Simple Object Access Protocol):**

- SOAP is a more rigid protocol that requires XML based messaging for communication. Strict and secure protocol using XML for structured messaging.
- Data Format: XML

❖ **GraphQL:**

- Modern query language that lets clients fetch only the data they need.
- Data Format: JSON

❖ **gRPC:**

- High performance framework using Protocol Buffers (Protobuf).
- Data Format: Binary

Representational State Transfer (REST)

- ❖ REST stands for **Representational State Transfer**
- ❖ The constraints of REST architecture allowing interaction with RESTful web services.
- ❖ It defines a set of functions (**GET, PUT, POST, DELETE**) that clients use to access server data.
 - **GET** (retrieve a record)
 - **PUT** (update a record)
 - **POST** (create a record)
 - **DELETE** (delete the record)
- ❖ Its main feature is that REST API is **stateless**, i.e., the servers do not save clients' data between requests.

Core Principles of REST

❖ **Statelessness:**

- The server does not "remember" the client's previous interactions.

❖ **Client-Server Separation:**

- There is a clear separation of concerns.
- The Client (browser/mobile app) handles the user interface and initiates requests, while the Server handles data storage, processing, and sends responses

❖ **Uniform Interface**

- APIs should use a standard, consistent way of communicating.
- This usually means relying on the standard **HTTP protocol** (GET, POST, etc.) rather than inventing new rules.
- This makes the API scalable and easier to maintain.

RESTful API Design Guidelines

❖ Resource Modeling (Naming Conventions)

- **Use Nouns, Not Verbs:** URIs should name the thing (resource), not the action. The action is defined by the HTTP method (GET, POST, etc.).
 - **Bad:** /getUsers or /updateUser
 - **Good:** /users
- **Use Plural Nouns:** Use plural names for collections of resources.
 - Example: /users (Collection) vs. /users/:id (Single resource) .
- **Use Lowercase:** Always use lowercase letters in URIs.
- **Use Hyphens:** Use hyphens (-) to separate words, not underscores or CamelCase.
 - Example: /user-profile instead of /userProfile.
- **Versioning:** Always version your API to manage changes without breaking client applications.
 - Example: /v1/users.

❖ HTTP Methods (The Verbs)

- GET (retrieve a record)
- PUT (update a record)
- POST (create a record)
- DELETE (delete the record)

RESTful API Design Guidelines

❖ Standard Status Codes

➤ Success:

- **200 OK:** Everything is fine.
- **201 Created:** A new resource was successfully created.

➤ Client Errors (User's Fault):

- **400 Bad Request:** Invalid payload or input.
- **401 Unauthorized:** Credentials are missing or incorrect .
- **403 Forbidden:** You are logged in but don't have permission.
- **404 Not Found:** The URL or resource does not exist.
- **429 Too Many Requests:** Rate limiting exceeded.

➤ Server Errors (API's Fault):

- **500 Internal Server Error:** Syntax error or server crash.

Advanced Design Topics

❖ Filtering and Pagination

- Use **Query Parameters** (the part of the URL after the ?) to **filter, sort, or paginate** data.
- This allows clients to request only the specific data they need, improving performance.
- **Example: ?page=1&sortBy=name&sortDir=asc.**

❖ Hypermedia

- Include links to related resources within the API response to help discoverability.
- Example: Returning a link to a user's photo (**`https://.../images/1.jpeg`**) inside the user object .

❖ Optimizing Performance

- **Compression:** Use compression techniques to reduce data size.
- **Nested Objects:** Return full resource objects instead of just IDs to prevent the client from making multiple extra calls.
 - **Bad:** Returning **"authorId": 1** (forces another call to get author name) .
 - **Good:** Returning **"author": { "id": 1, "name": "Author Name" } .**

JSON (JavaScript Object Notation)

- ❖ JSON is a lightweight **data-interchange format** based on a subset of the JavaScript programming language .
- ❖ It is designed to be **easy** for humans to **read and write**, while also being easy for machines to parse and generate.
- ❖ Syntax Structure
 - **Objects:** Data is enclosed in curly braces {} to define an object.
 - **Key-Value Pairs:** Data is stored as pairs, where a field name (key) is associated with a value.
 - Example: {"name": "username"}.

JSON

```
{  
  "postId": 1,  
  "title": "some post title",  
  "content": "post contend",  
  "author": {  
    "id": 2,  
    "name": "author name"  
  }  
}
```

Hypermedia Example

JSON

```
{  
  "name": "username",  
  "photo": {  
    "fileName": "1.jpeg",  
    "link": "https://storage.example.com/images/1.jpeg"  
  }  
}
```

XML (eXtensible Markup Language)

- ❖ XML stands for eXtensible Markup Language.
 - Definition: It is a **markup language** designed to **store and transport data**.
 - Role in API: Like JSON, it is a **standard format for API payloads (requests and responses)**. It allows different systems to exchange information.
 - "Extensible": Unlike HTML (where tags like <h1> are fixed), in XML, developers define their own tags (e.g., <user>, <price>).
- ❖ XML Structure & Example
 - XML uses a tree structure consisting of nested tags.
 - **Tags:** Data is wrapped in start <tag> and end </tag> markers.
 - **Root Element:** Every XML document must have one single parent "root" element that wraps everything.
 - **Attributes:** Metadata added inside the opening tag (e.g., id="101").

XML

```
<user>
  <id>101</id>
  <name>John Doe</name>
  <email>john@example.com</email>
  <roles>
    <role>Admin</role>
    <role>Editor</role>
  </roles>
  <isActive>true</isActive>
</user>
```

Parsing

"Parsing" is the process of converting the raw text received from the API into a usable object in the programming language.

Raw Data: **'{"title": "API Design", "price": 29.99}'** → It's in string

For Json

```
// The API sends a string
const jsonString = '{"title": "API Design", "price": 29.99}';

// We parse it into an object with ONE line
const book = JSON.parse(jsonString);

// We use the data immediately
console.log(book.title); // Output: API Design
```

Parsing

Raw Data: **'{"title": "API Design", "price": 29.99}'** → It's in string

For XML

```
// The API sends a string
const xmlString = '<book><title>API Design</title><price>29.99</price></book>';

// We need a special parser tool
const parser = new DOMParser();
const xmlDoc = parser.parseFromString(xmlString, "text/xml");

// We have to find the specific tag manually to get the data
const title = xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;

console.log(title); // Output: API Design
```

Serialization & Deserialization

❖ Serialization (Server → Client):

- Definition: This is the process of **converting complex data structures** (like a user object in a database or a class instance) **into a format that can be easily transmitted over the internet** (like a **JSON string**) .
- Role in API: When the server sends an **API Response**, it must **"serialize" the data** found in the database into a JSON payload so the browser can read it .
- Example: The document shows converting **raw data into a structured JSON response** to optimize performance .

❖ Deserialization (Client → Server):

- Definition: This is the reverse process. It takes the **"raw text" (payload) sent by the client and converts it back into a complex object or data structure** that the server's code can understand and manipulate.
- Role in API: When a **client sends a POST or PUT request with data**, the server **"deserializes"** that JSON body to save it to the database.

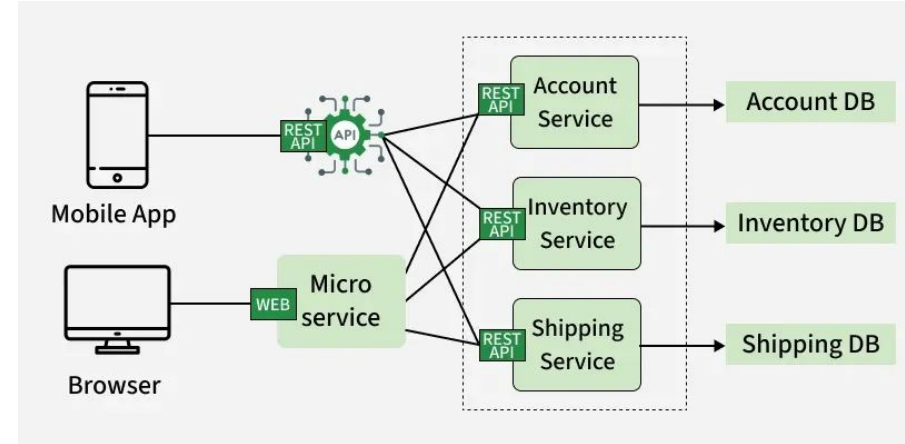
Data Validation

Before the server accepts deserialized data (e.g., to create a new user), it must **verify that the data is correct and safe.**

- ❖ What is it? The process of **checking the Payload (input data)** against a **set of rules.**
- ❖ Why is it needed? To **ensure** the data follows the **correct format before processing.** For example, **ensuring an email field** actually contains an email address.
- ❖ Handling Failure:
 - If validation fails, the API stops immediately and returns an error.
 - Status Codes: The document highlights 400 Bad Request for invalid payloads.
 - Specific Validation Errors: It also details a specific error response format (Status 422) for validation issues, providing details on what went wrong .

Microservices Architecture

- ❖ Microservices are an architectural approach to developing software applications **as a collection of small, independent services** that communicate with each other over a network.
- ❖ Components:
 - **Microservices**
 - Microservices are independent, loosely coupled services designed around specific business functions.
 - Handle a single, well-defined capability.
 - Can be developed and deployed independently.



Microservices Architecture

❖ API Gateway

- The API Gateway serves as a centralized entry point for all external client requests.
 - Manages request routing and authentication
 - Forwards requests to appropriate microservices

❖ Service Registry and Discovery

- Service Registry and Discovery keeps track of available services and their locations.
 - Stores service network addresses.
 - Enables dynamic inter-service communication.

❖ Load Balancer

- A Load Balancer distributes incoming traffic across service instances.
 - Improves availability and reliability.
 - Prevents service overload.

❖ Containerization

- Containerization packages microservices and their dependencies into containers.
 - Docker encapsulates services consistently
 - Kubernetes manages scaling and orchestration

❖ Event Bus / Message Broker

- An Event Bus or Message Broker enables asynchronous communication between services.

Microservices Architecture

❖ Database per Microservice

- In the Database per Microservice pattern, each microservice owns and manages its own dedicated database to maintain data autonomy.

❖ Caching

- Caching improves performance by storing frequently accessed data closer to services.
 - Reduces database load
 - Decreases response latency

❖ Fault Tolerance and Resilience

- Fault tolerance and resilience mechanisms enable the system to continue functioning even when some components fail.
 - Uses techniques such as circuit breakers, retries, and fallbacks.
 - Maintains overall system stability and availability.

Loose Coupling in Microservices

❖ Definition:

- Loose coupling means system components (services) operate independently and do not rely heavily on one another's internal workings.
- Changes to one service do not break the others or the client application.

❖ Role of API Gateway:

- Achieved by using an API Gateway as an intermediary between the client and the backend services.
- The Gateway acts as a "Gatekeeper" or proxy, ensuring the client never talks directly to the backend.

❖ Client Independence:

- Clients only need to know the Gateway's endpoint, not the complex structure of the backend services.
- Backend services can be updated, moved, or replaced without requiring changes to the client app.

❖ Decoupled Logic:

- Common tasks like Authentication, Rate Limiting, and Usage Metrics are offloaded to the Gateway.
- This keeps individual services focused purely on their specific business logic, rather than security or traffic management.

API Endpoints

- ❖ **Definition:** An endpoint is a specific URL that corresponds to a resource or set of resources.
 - Structure: It combines an HTTP method with a URI to define the action.
 - Example: GET /users to retrieve a list, POST /users to create a new user .
 - Best Practice: Use plural nouns for collections (e.g., /users) and avoid using verbs in the URL (e.g., avoid /getUsers) .
- ❖ **Returning JSON**
 - Body: The actual data requested (the payload).
 - Status Code: A numerical code indicating success (200 OK, 201 Created) or failure (400 Bad Request, 404 Not Found) .
 - Headers: Metadata about the response, such as Content-Type: application/json
- ❖ **Why API Testing is Critical**
 - The Challenge: API testing is considered one of the most challenging types of testing.
 - The Risk: Missing test cases can lead to significant problems in production after full integration.
 - Debugging: Issues found in production are much harder to debug than those found during initial testing.

API Testing Tools

Postman: Manual and automated API testing

SoapUI: SOAP & REST API testing

JMeter: Load and performance testing

Apigee: Enterprise API management

vREST: Automated regression testing